DROS-6P: A Deterministic Runtime Governance Architecture for Six Trust Boundaries in Enterprise AI Agents

Anonymous Authors

Abstract—Autonomous AI agents are increasingly entrusted with sensitive enterprise workflows but operate with probabilistic reasoning over untrusted contexts. Traditional defenses relying on identity or application-layer filtering fail to distinguish authorized from compromised behavior once malicious actions originate from valid credentials, creating a fundamental semantic-execution governance gap. This paper introduces DROS-6P, a deterministic in-band runtime governance architecture organized around six complementary trust boundaries: Principal identity (P_1), Authorization (P_2), Tool/Action bounds (P_3), Policy gating (P_4), tamper-evident Auditing (P_5), and Expiry/Revocation (P_6). The architecture separates agent intent from execution authority and mediates protected operations through a binary C-ABI enforcement boundary. Capability authorization uses constant-time 64-bit bitmask evaluation, while dynamic policy revocation employs atomic Read-Copy-Update (RCU) state-pointer transitions ($T_{\text{swap}} \approx 420$ ns). We formalize two execution governance invariants: unauthorized-execution containment and execution-to-evidence completeness. In a 72-hour continuous soak evaluation comprising 160,611 total requests (including 137,751 policy enforcement probes and 22,860 benign controls), the system exhibited a median decision latency of 26.21 μs and a P99 latency of 242.69 μs. Across predefined adversarial scenarios and six heterogeneous application tracks, no unauthorized physical state transition was observed within the evaluated coverage space. These results support in-band deterministic mediation as a practical foundation for runtime governance of enterprise AI agents.

Index Terms—AI Agent Security, Runtime Governance, Six Trust Boundaries, C-ABI Boundary Enforcement, Read-Copy-Update (RCU), Execution-to-Evidence Invariants.

I. Introduction

Autonomous AI agents are increasingly entrusted with sensitive enterprise operations, including financial ledger management, customer relationship handling, and automated software deployment. Unlike traditional deterministic software services, these agents dynamically generate tool invocations based on probabilistic large language model (LLM) reasoning over untrusted external context.

A. The Semantic-Execution Governance Gap

Modern enterprise security architectures face a fundamental split when governing autonomous agent workloads:

1) Application-Level Semantic Firewalls: Prompt inspection filters, output classifiers, and JSON schema validators evaluate natural language tokens but lack execution-boundary containment. Interpreter escapes, obfuscated parameters, and multi-turn context poisoning consistently circumvent these probabilistic filters.
2) Kernel-Level Sandboxes: Low-level OS primitives (e.g., Linux Seccomp, Namespaces, eBPF) enforce binary system-call filtering but lack application-layer semantic context. They cannot determine whether a database write originated from an authorized Finance Agent or a hijacked Support Agent executing within the same shared runtime worker process.

When an agent is compromised via Indirect Prompt Injection (IPI) or goal hijacking, the adversary operates from within an authenticated security boundary—rendering traditional IAM and perimeter defenses context-blind. We term this the agent-to-execution attribution gap: the inability to map an application-level agent identity to its corresponding OS-level system call stream.

B. Research Thesis and Contributions

The core thesis of DROS-6P is that effective runtime governance requires architectural co-location of six previously separated trust boundaries directly at the execution boundary, where authorization decisions can deterministically constrain physical execution and emit verifiable cryptographic evidence.

This paper makes three primary contributions:

1) Six Trust Boundaries Model: We formulate a formal runtime trust-boundary model (P_1–P_6) closing the six essential questions of agent execution, structured as the DROS-6P framework.
2) Deterministic In-Band Substrate: We present a runtime architecture that decouples agent intent from execution authority, enforcing capability constraints at a binary C-ABI boundary using $O(1)$ bitmask evaluations and atomic RCU pointer swaps ($T_{\text{swap}} \approx 420$ ns).
3) Empirical Invariant Evaluation: We evaluate the architecture under a 72-hour continuous soak workload (160,611 requests) and six heterogeneous enterprise tracks, verifying zero unauthorized physical executions and complete audit trail integrity within the evaluated coverage space.

II. The Six Trust Boundaries (P1–P6)

To provide comprehensive execution governance, an agent substrate must continuously mediate every consequential action across six distinct and complementary trust boundaries, which collectively provide a structured operational baseline for accountable autonomous agents:

- P1: Principal Identity (Who is acting?): Cryptographic binding between the autonomous agent instance and an ephemeral identity token (DrosIdentityToken, DIT) containing an agent identifier and Ed25519 signature.
- P2: Authorization (What is permitted?): A 64-bit capability bitmask encoding permitted operation classes, resolving the context-blindness of general-purpose runtime workers.
- P3: Tool/Action Bound (Which tool and parameters?): Canonicalization and parameter validation of tool invocation requests against declarative schema bounds.
- P4: Policy Gate (Under what policy?): In-band Policy Decision Point (PDP) and Policy Enforcement Point (PEP) executing deterministic boundary checks.
- P5: Audit Log (What verifiable evidence exists?): Sequential SHA-256 hash-chained execution logs providing cryptographically verifiable integrity and tamper evidence.
- P6: Expiry & Dynamic Revocation (Is authority still valid?): Epoch-bounded token lifetimes combined with lock-free RCU atomic pointer invalidation for bounded-latency policy-state transitions.

III. Deterministic Runtime Enforcement Architecture

A. Decoupled Intent and Execution Planes

DROS-6P enforces a strict separation between the Intent Plane (where the LLM reasons and formats tool calls) and the Execution Plane (where physical syscalls, network sockets, and database writes occur). The application worker process cannot directly invoke protected system resources; every request must traverse the binary C-ABI gate. This decoupling ensures that even if the Intent Plane is fully compromised, the Execution Plane cannot be coerced without valid capability authorization.

B. C-ABI Capability Bitmask Evaluation ($O(1)$)

To prevent latency amplification in multi-agent environments, authorization policies are compiled into immutable 64-bit bitmasks. The capability-checking step executes via constant-time bitwise operations:

$$\text{Dec} = \begin{cases} \text{PERMIT}, & \text{if } (M_{\text{agent}} \wedge M_{\text{req}}) = M_{\text{req}} \\ \text{DENY}, & \text{otherwise} \end{cases} \tag{1}$$

If the requested capability bit is not asserted, the enforcement point immediately halts execution. The isolated binary denial-path primitive was measured at less than 500 ns, excluding ingress parsing, token verification, audit-chain processing, and protocol adaptation (which maps to HTTP 403 at the protocol adapter).

C. Atomic RCU State Revocation ($T_{\text{swap}} \approx 420$ ns)

Dynamic policy updates and emergency security revocations execute via Read-Copy-Update (RCU). The runtime maintains an active pointer to the capability state structure. When a revocation signal occurs, the management thread allocates a mutated copy, updates the bitmask, and executes an atomic pointer exchange:

$$\text{AtomicPtr.swap}(\&P_{\text{act}}, P_{\text{new}}, \text{Ordering::Release}) \tag{2}$$

The measured pointer-swap primitive latency was approximately 420 ns. After the atomic state transition is linearized, subsequent authorization checks observe the updated policy state.

D. Dual Execution Governance Invariants

Let $Auth_E(x)$ be the effective authorization decision derived from the authenticated principal, capability state, tool bounds, and policy state ($Auth_E(x) = f(P_1, P_2, P_3, P_4, P_6)$). Note that P_5 (Audit Log) is intentionally excluded from $Auth_E$ because auditing serves as an evidence boundary rather than an authorization input; it instead constrains the evidentiary completeness of successful execution.

The substrate formally enforces two complementary mathematical invariants over the evaluated coverage space X_{covered}:

$$\forall x \in X_{\text{covered}}, \quad Auth_E(x) = \text{DENY} \implies Exec(x) = 0 \tag{3}$$

$$\forall x \in X_{\text{covered}}, \quad Exec(x) = 1 \implies Audit(x) = 1 \tag{4}$$

Here, $Exec(x) = 0$ denotes that the governed action produced zero physical execution side effects. $Audit(x) = 1$ denotes that the execution event is committed to the cryptographically chained audit structure and passes integrity verification.

IV. Experimental Methodology and Measurement

A. Evaluation Environment and Apparatus

All benchmarks were conducted within an isolated containerized evaluation environment (Table I).

B. Measurement Boundary Definition

To ensure rigorous reporting, we define the end-to-end decision latency t_{decision} explicitly across its constituent stages:

$$t_{\text{decision}} = t_{\text{in}} + t_{\text{DIT}} + t_{\text{mask}} + t_{\text{audit}} + t_{\text{out}} \tag{5}$$

Measurements were captured using high-resolution monotonic clocks (perf_counter_ns) across 160,611 continuous requests distributed across concurrency levels of 1, 5, 10, 25, and 50 worker threads.

TABLE I
Evaluation Hardware and System Configuration

Parameter	Specification
Host Processor	Intel(R) Xeon(R) CPU E3-1275L v3 @ 2.70GHz
Host Memory	16 GB Dual-Channel DDR3 RAM
Host OS	Windows 10 Enterprise LTSC (Build 19044)
Guest VM	Ubuntu 22.04 LTS (Kernel 5.15 / Docker 24.0)
Engine	DROS-6P In-Band Daemon (Multi-threaded C-ABI)
Container	Python 3.11 Runtime / Isolated Linux Container

V. Empirical Results and Stress Evaluation

A. 72-Hour Continuous Soak Evaluation

To evaluate long-duration stability and memory safety, DROS-6P was subjected to a 72-hour continuous multi-scenario workload comprising 160,611 total requests. Of these, 137,751 were evaluated against unauthorized-action policies, while the remaining 22,860 consisted of benign and control-path workloads (Table II).

TABLE II
72-Hour Continuous Soak Evaluation Telemetry ($N = 160,611$)

Category	Measured Parameter	Observed Value
Latency	End-to-End Decision (P50)	26.21 μs
	End-to-End Decision (P95)	38.45 μs
	End-to-End Decision (P99)	242.69 μs
	Binary Denial Primitive	$<$ 500 ns
Stability	Resident Set Size (RSS) Growth	0 MB
	Process Panic / Crash Count	0
Invariants	Unauthorized Physical Actions	$0/137,751$
	Audit Hash Chain Integrity	100%

Throughout the 72-hour evaluation, no observable RSS growth was detected due to pre-allocated zero-heap ring buffers. Under concurrency spikes up to 50 threads, P99 latency remained bounded at 242.69 μs, while the median decision latency settled at 26.21 μs.

B. Direct-Execution Comparative Baseline Suite

We evaluated five core security scenarios (ATS-001 through ATS-005) comparing DROS-6P against a direct-execution baseline without runtime enforcement (Table III). Under the evaluated baseline configuration, all five scenarios resulted in unauthorized physical state transitions in every test run, whereas DROS-6P completely contained all observed attempts.

C. Heterogeneous Industry Application Tracks

The architecture was further evaluated across six representative enterprise application tracks to demonstrate applicability of the DROS-6P model across heterogeneous domains (Table IV).

TABLE III
Comparative Security Containment Matrix

Scenario	Threat Description	Baseline	DROS-6P
ATS-001	Indirect Prompt Injection (IPI)	100% Breach	0% Breach
ATS-002	Sensitive Data Exfiltration	100% Breach	0% Breach
ATS-003	Unauthorized Deployment Push	100% Breach	0% Breach
ATS-004	B2B Supply Chain Poisoning	100% Breach	0% Breach
ATS-005	Post-Compromise Syscall Escape	100% Breach	0% Breach

TABLE IV
Heterogeneous Industry Governance Scenario Matrix

Track	Target Boundary	Enforcement Mechanism	Breaches
Mfg	BOM IP Protection	Dynamic Redaction	0 / 100
Finance	Anti-Money Laundering	Policy Gate / Approval	0 / 100
Health	PHI / Medical Privacy	DIT Field Masking	0 / 100
Gov	Privilege Escalation	C-ABI Bitmask Gate	0 / 100
Fintech	Account Takeover	Atomic Revocation	0 / 100
Supply	B2B Cross-Tenant	Signed DIT Attestation	0 / 100

Across 600 total instantiated test runs across all six domains, no unauthorized physical state transitions were observed under the evaluated scenarios.

VI. Related Work

Runtime security for autonomous workloads spans several established and emerging research domains:

- LLM Guardrails and Semantic Filtering: Systems such as NVIDIA NeMo Guardrails [1] and Microsoft Agent Framework [2] inspect prompt text and tool arguments. While effective for semantic moderation, they remain probabilistic and do not enforce binary execution boundaries.
- Capability-Based Access Control: Classical capability systems [4], [5] associate execution privileges with unforgeable tokens. DROS-6P extends capability theory to autonomous agent swarms by compiling dynamic permissions into constant-time 64-bit bitmasks.
- System Call Interception and Kernel Tracing: Systems such as AgentSight [3] track low-level system calls via eBPF. However, pure OS tracing mechanisms lack user-space agent principal context, making it difficult to attribute high-level intent to specific agent identities.
- Information Flow Control and Provenance: Architectures such as TaintDroid [6] enforce dynamic data propagation boundaries, while provenance frameworks [7] capture execution histories. DROS-6P uni-

fies data boundary redaction with hash-chained execution logs.

- Zero-Trust Workload Identity: Standards such as SPIFFE/SPIRE [8] and NIST Zero Trust Architecture [9] provide workload identity attestations. DROS-6P adapts these principles to ephemeral LLM tool execution lifecycles.

While existing systems typically address subsets of these concerns at different architectural layers, DROS-6P colocates the six trust boundaries (P_1–P_6) directly at the point where agent requests become physically consequential operations, ensuring deterministic enforcement and audit-chain integrity.

VII. Conclusion

This paper presented DROS-6P, an in-band deterministic runtime governance architecture that closes the six essential trust boundaries of autonomous enterprise AI agents. By enforcing 64-bit capability bitmasks at a binary C-ABI boundary and providing sub-microsecond atomic state revocation via RCU pointer transitions ($T_{\text{swap}} \approx$ 420 ns), DROS-6P bridges the semantic-execution governance gap. Empirical evaluations over 160,611 continuous requests demonstrated zero unauthorized executions within the evaluated coverage space and 100% audit trail integrity. The architecture is not a replacement for semantic guardrails or application-level policies but serves as a complementary enforcement substrate that provides deterministic physical containment. Future work will explore integration with hardware security extensions (e.g., Intel SGX, ARM TrustZone) and formal verification of the C-ABI enforcement path.

AI Declaration

Generative AI tools were used solely for limited technical assistance, including language polishing, grammatical refinement, and LaTeX typesetting assistance. The conceptual framework, research questions, system architecture, formal invariants, experimental design, interpretation of results, and final claims were determined and verified by the authors. The authors take full responsibility for the final content of this paper.

References

[1] NVIDIA, "NeMo Guardrails: Programmable Guardrails for LLM Applications," NVIDIA Developer Documentation, 2024.

[2] Microsoft, "Microsoft Agent Framework Documentation: Tool Calling and Execution Governance," Microsoft Learn, 2025.

[3] X. Zhang et al., "AgentSight: eBPF-Powered Tracing and Context Correlation for Autonomous LLM Agents," arXiv preprint arXiv:2408.01234, 2024.

[4] J. B. Dennis and E. C. Van Horn, "Programming Semantics for Multiprogrammed Computations," Communications of the ACM (CACM), vol. 9, no. 3, pp. 143–155, 1966.

[5] H. M. Levy, Capability-Based Computer Systems, Digital Press, 2014.

[6] W. Enck et al., "TaintDroid: An Information-Flow Tracking System for Real-Time Privacy Monitoring on Smartphones," ACM Transactions on Computer Systems (TOCS), vol. 32, no. 2, pp. 1–32, 2014.

[7] K.-K. Muniswamy-Reddy et al., "Provenance-Aware Storage Systems," in Proc. USENIX Annual Technical Conference (ATC), 2006, pp. 43–56.

[8] Cloud Native Computing Foundation (CNCF), "SPIFFE: Secure Production Identity Framework for Everyone," CNCF Standard Specification, 2020.

[9] NIST, "Zero Trust Architecture," NIST Special Publication 800-207, 2020.

[10] P. E. McKenney, "Is Parallel Programming Hard, And, If So, What Can You Do About It? (Read-Copy Update Architecture)," Linux Technology Center, IBM, 2024.

[11] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," OWASP Standard, 2025.

[12] European Parliament, "Artificial Intelligence Act (Regulation EU 2024/1689), Article 50: Transparency and Traceability of AI Systems," Official Journal of the European Union, 2024.